Generated from `docs/handoff-guides/markdown/blueprint-schema.md` by `tools/guide-publisher`.

Blueprint Bundle Handoff Guide

Use this guide when you need to describe the **whole Blueprint as one bundle**: what system is being modeled, how the content is organized, what slices exist, what is shared, and what bundle- level principles or migrations matter.

This guide is about the **shape of the whole model**, not the details of one layer.

KNOWLEDGE AREA

This guide helps with the **whole blueprint setup and scope**.

Guide family map

Where this guide sits in the full Blueprint Handoff Atlas family.

ROOT & CROSS-CUTTING

Handoff Atlas · **Blueprint Bundle** · Metamodel Vocabulary · Migrations

DESIGN

Architecture · Concepts · Domain · Dynamics · Infrastructure · Interactions · Models · Quality · Rules · Story

GOVERNANCE

Capability · Decisions · Motivation · Organization · Roadmap · Test Cases · Value Stream

Capture → focus → transformation

What to gather first, what this layer focuses on, and what a modeler or AI agent can produce from it.

Capture inputs

- System scope
- Slices & shared files
- Bundle conventions

→

Blueprint Bundle

This guide helps non-technical people express this layer clearly.

→

Transformation outputs

- Blueprint layout
- Root vs slice structure
- Whole-model clarity

What belongs here

- the name and overall scope of the system being modeled
- the system-level description
- the overall Blueprint layout
- the slices or major domains inside the Blueprint
- which files are shared at root level
- bundle-level principles, conventions, and migrations

What does not belong here

- detailed layer content for one specific design or governance file
- low-level schema or YAML mechanics
- detailed process, UI, or rule descriptions that belong in a specific guide

Core things to capture

- what system or product the Blueprint describes
- where its important boundaries are
- whether it is modeled as one unified bundle or as slices
- what the slices mean in business terms
- what shared/system-level material exists at the root
- what bundle-level principles or conventions matter
- whether planned migrations affect the overall Blueprint shape

Core relationships to capture

- Blueprint bundle -> slices/domains
- Blueprint bundle -> shared root files
- slice -> business domain/capability it represents

- bundle -> principles/conventions/migrations that affect all layers

Modus operandi

Describe the Blueprint from outside in:

1. name the overall system
2. explain what is in and out of scope
3. decide whether the model is unified or sliced
4. describe each slice in business language
5. describe what must stay shared at the root
6. note conventions or migrations that affect the whole bundle

Prompt set

- what system are we modeling?
- what are the major domains or slices?
- what belongs at the root because it is shared?
- where are the important boundaries?
- what conventions or principles govern the whole Blueprint?
- what planned migrations change the whole structure?

Free-text intake template

- **Blueprint/system name:**
- **Overall purpose:**
- **In-scope / out-of-scope:**
- **Layout mode (unified or slices):**
- **Slices/domains:**
- **Shared root-level content:**
- **Bundle-level principles/conventions:**
- **Bundle-level migration notes:**

Worked example

- **Blueprint/system name:** Storefront Commerce Platform
- **Overall purpose:** model shopping, checkout, and post-purchase workflows and the supporting governance context
- **Layout mode:** slices
- **Slices/domains:** catalog, checkout, post-purchase

- **Shared root-level content:** overall architecture, organization, shared motivation, common decisions
- **Bundle-level principles:** single source of truth for order identity, policy-based automation within guardrails
- **Bundle-level migration notes:** move from one unified commerce slice to separate checkout and post-purchase slices

Layer-specific handoff guidance

When handing off Blueprint Bundle notes, make sure the receiver can answer:

- what the **whole Blueprint** is modeling
- what belongs at the **root** versus in a **slice**
- which conventions apply across the entire bundle

The handoff is strongest when it includes a short system summary, a slice list with business meaning, and clear statements about what is shared across the whole Blueprint.

How this becomes YAML

A technical modeler or AI agent can transform this into bundle-level Blueprint fields such as name, description, layout, slices, shared file locations, constitution/conventions, and migration links.

Common mistakes

- treating the Blueprint bundle like just another layer file
- naming slices without explaining what business area each slice represents
- putting shared content into one slice by accident
- mixing whole-bundle structure with detailed layer semantics